

```

# -*- coding: utf-8 -*-
"""
Code to train and evaluate TF-IDF+LogReg and Transformer models
for overall sentiment classification based on star ratings.
"""

# 1. Install and import NLTK
!pip install nltk
import nltk
# Import necessary libraries
import pandas as pd
import numpy as np
import json
import re
import time
import os
from datetime import datetime

# Text Preprocessing
import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from nltk.stem import WordNetLemmatizer

# Scikit-learn for baseline models and metrics
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score,
    precision_recall_fscore_support,
    classification_report,
    confusion_matrix,
    ConfusionMatrixDisplay
)

# PyTorch and Hugging Face Transformers
import torch
from torch.utils.data import Dataset, DataLoader
from torch.optim import AdamW
from transformers import (
    AutoTokenizer,
    AutoModelForSequenceClassification,
    get_linear_schedule_with_warmup,
    DistilBertTokenizer, # Using DistilBERT as in the original notebook
    DistilBertForSequenceClassification
)

# Plotting (Optional, for confusion matrix)
import matplotlib.pyplot as plt
import seaborn as sns

import gc

# 2. Modified download function
def download_nltk_data():
    """Downloads necessary NLTK data packages."""
    packages = ['punkt', 'stopwords', 'wordnet', 'punkt_tab'] # Added 'punkt_tab' to the list
    for package in packages:
        try:
            nltk.data.find(f'corpora/{package}' if package != 'punkt' else f'tokenizers/{package}')
        except LookupError:
            print(f"Downloading NLTK package: {package}...")
            nltk.download(package, quiet=True)
    print("NLTK data check complete.")

# 3. Call the download function
download_nltk_data()

# --- Configuration & Setup ---

# Set random seed for reproducibility
SEED = 42
np.random.seed(SEED)
torch.manual_seed(SEED)
if torch.cuda.is_available():
    torch.cuda.manual_seed_all(SEED)

```

```

# Check for GPU
device = torch.device('cuda') if torch.cuda.is_available() else torch.device('cpu')
print(f"Using device: {device}")

# --- Download NLTK data (if needed) ---
def download_nltk_data():
    """Downloads necessary NLTK data packages."""
    packages = ['punkt', 'stopwords', 'wordnet']
    for package in packages:
        try:
            nltk.data.find(f'corpora/{package}' if package != 'punkt' else f'tokenizers/{package}')
        except LookupError:
            print(f"Downloading NLTK package: {package}...")
            nltk.download(package, quiet=True)
    print("NLTK data check complete.")

download_nltk_data()

# --- File Paths & Data Loading ---
# Adjust paths based on whether running in Colab or locally
try:
    from google.colab import drive
    drive.mount('/content/drive')
    # Adjust Google Drive paths as needed
    review_file_path = '/content/drive/My Drive/Colab Notebooks/SentimentModels_Stars/yelp_academic_dataset_review.json'
    business_file_path = '/content/drive/My Drive/Colab Notebooks/SentimentModels_Stars/yelp_academic_dataset_business.json'
    # Define output directory for models if needed
    output_dir = '/content/drive/My Drive/Colab Notebooks/SentimentModels_Stars'
    print("Running in Colab environment.")
except ModuleNotFoundError:
    # Adjust local paths as needed
    review_file_path = r"G:\My Drive\CMPS 6730 NLP\FinalProject\yelp_academic_dataset_review.json"
    business_file_path = r"G:\My Drive\CMPS 6730 NLP\FinalProject\yelp_academic_dataset_business.json"
    # Define output directory for models if needed
    output_dir = './SentimentModels_Stars'
    print("Running in local environment.")

# Create output directory if it doesn't exist
os.makedirs(output_dir, exist_ok=True)
print(f"Model output directory: {output_dir}")

def load_data(review_path, business_path):
    """Loads review and business data, filters, and merges."""
    print(f"\nLoading review data from: {review_path}")
    start_time = time.time()
    try:
        # Load reviews in chunks for memory efficiency
        all_data = []
        chunksize = 100000
        for chunk in pd.read_json(review_path, lines=True, chunksize=chunksize):
            all_data.append(chunk[['review_id', 'business_id', 'stars', 'text', 'date']]) # Select columns early
        df_reviews = pd.concat(all_data, ignore_index=True)
    except Exception as e:
        print(f"Error loading review data: {e}")
        return None, None
    loading_time = time.time() - start_time
    print(f"Review data loaded in {loading_time:.2f} seconds. Shape: {df_reviews.shape}")

    print(f"\nLoading business data from: {business_path}")
    start_time = time.time()
    try:
        # Handle potential JSON errors in business data
        business_data = []
        with open(business_path, 'r', encoding='utf-8') as f:
            for line in f:
                try:
                    business_data.append(json.loads(line))
                except json.JSONDecodeError:
                    print(f"Skipping malformed line in businesses")
                    continue
        df_business = pd.DataFrame(business_data)
        df_business = df_business[['business_id', 'categories']] # Select only needed columns
    except Exception as e:
        print(f"Error loading business data: {e}")
        return df_reviews, None # Return reviews even if businesses fail
    loading_time = time.time() - start_time
    print(f"Business data loaded in {loading_time:.2f} seconds. Shape: {df_business.shape}")

```

```

print(f"Business data loaded in {loading_time:.2f} seconds. Shape: {df_business.shape} ")

# Convert review date
df_reviews['date'] = pd.to_datetime(df_reviews['date'])

# Filter businesses for Restaurants/Food
df_business['categories'] = df_business['categories'].fillna('')
target_restaurants = df_business[
    df_business['categories'].str.contains('Restaurant|Food', case=False, regex=True)
].copy()
target_business_ids = set(target_restaurants['business_id'])
print(f"\nFound {len(target_business_ids)} unique target restaurant business IDs.")

if not target_business_ids:
    print("Warning: No restaurant business IDs found. Check categories.")
    # Decide how to proceed - here we continue with only date filtering
    # return df_reviews, None

# Filter reviews by date (e.g., 2019-2024)
start_date = datetime(2019, 1, 1)
end_date = datetime(2024, 12, 31)
df_reviews_filtered_date = df_reviews[
    (df_reviews['date'] >= start_date) & (df_reviews['date'] <= end_date)
].copy()
print(f"Reviews filtered by date ({start_date.date()} to {end_date.date()}). Shape: {df_reviews_filtered_date.shape}")

# Filter reviews by target business IDs
if target_business_ids:
    df_filtered = df_reviews_filtered_date[
        df_reviews_filtered_date['business_id'].isin(target_business_ids)
    ].copy()
    print(f"Reviews filtered by target business IDs. Shape: {df_filtered.shape}")
else:
    print("Skipping business ID filtering as no target IDs were found.")
    df_filtered = df_reviews_filtered_date # Use date-filtered if no business match

if df_filtered.empty:
    print("\nWARNING: No reviews match the filtering criteria. Cannot proceed.")
    return None, None

print(f"\nFinal shape for processing: {df_filtered.shape}")
# Select final columns needed
df_processed = df_filtered[['review_id', 'text', 'stars']].copy()
print(df_processed.head())

# Optional: Cleanup memory
del df_reviews, df_business, df_reviews_filtered_date, target_restaurants, df_filtered
import gc
gc.collect()
print("\nIntermediate dataframes cleaned up.")

return df_processed

# Load the data
# Load the data
df_processed = load_data(review_file_path, business_file_path) # Get the single returned DataFrame
if df_processed is None or df_processed.empty:
    print("Exiting due to data loading/filtering issues.")
    # exit() # Uncomment to stop execution if needed

# --- Text Preprocessing Function ---
lemmatizer = WordNetLemmatizer()
stop_words_set = set(stopwords.words('english'))

def preprocess_text(text):
    """Cleans and preprocesses text data."""
    if not isinstance(text, str):
        return ""
    text = text.lower() # Lowercase
    text = re.sub(r'http\S+|www\S+|https\S+', '', text, flags=re.MULTILINE) # Remove URLs
    text = re.sub(r'@\w+|\#', '', text) # Remove mentions and hashtags
    text = re.sub(r'[\^\w\s]', '', text) # Remove punctuation
    text = re.sub(r'\d+', '', text) # Remove numbers
    tokens = word_tokenize(text) # Tokenize
    processed_tokens = [
        lemmatizer.lemmatize(word) for word in tokens
        if word not in stop_words_set and word.isalpha() and len(word) > 1 # Keep words > 1 char
    ]

```

```

        return ' '.join(processed_tokens)

# Apply preprocessing
if df_processed is not None and not df_processed.empty:
    print("\nStarting text preprocessing...")
    start_time = time.time()
    # Ensure 'text' column is string type, fill NaNs with empty string
    df_processed['text'] = df_processed['text'].astype(str).fillna('')
    df_processed['processed_text'] = df_processed['text'].apply(preprocess_text)
    processing_time = time.time() - start_time
    print(f"Text preprocessing completed in {processing_time:.2f} seconds.")

    # Display some examples
    print("\nOriginal vs Processed Text Examples:")
    for i in range(min(3, len(df_processed))):
        print(f"--- Example {i+1} ---")
        print("Original:", df_processed['text'].iloc[i][:200] + "...")
        print("Processed:", df_processed['processed_text'].iloc[i])
        print("-" * 20)

    # Drop rows where processed text became empty
    original_len = len(df_processed)
    df_processed.dropna(subset=['processed_text'], inplace=True)
    df_processed = df_processed[df_processed['processed_text'] != '']
    print(f"\nRemoved {original_len - len(df_processed)} rows with empty processed text.")
    print(f"Shape after cleaning empty processed texts: {df_processed.shape}")
else:
    print("Skipping preprocessing as df_processed is not available.")

# --- Create Target Variable from Stars ---
def map_stars_to_sentiment(stars):
    """Maps star rating to sentiment label: 0 (neg), 1 (neu), 2 (pos)."""
    if stars in [1, 2]:
        return 0 # Negative
    elif stars == 3:
        return 1 # Neutral
    elif stars in [4, 5]:
        return 2 # Positive
    else:
        return -1 # Should not happen with Yelp data, but good practice

if df_processed is not None and not df_processed.empty:
    print("\nMapping star ratings to sentiment labels...")
    df_processed['overall_sentiment_stars'] = df_processed['stars'].apply(map_stars_to_sentiment)
    # Verify mapping
    print("Value counts for 'overall_sentiment_stars':")
    print(df_processed['overall_sentiment_stars'].value_counts())
    # Drop any potential -1 mappings if they occurred
    df_processed = df_processed[df_processed['overall_sentiment_stars'] != -1]
    print(f"Shape after mapping stars: {df_processed.shape}")
else:
    print("Skipping star mapping as df_processed is not available.")

# --- Prepare Data for Models ---
if df_processed is not None and not df_processed.empty:
    X = df_processed['processed_text']
    Y = df_processed['overall_sentiment_stars']

    # Split data - Stratify by the new star-based sentiment
    print("\nSplitting data into training and testing sets...")
    X_train, X_test, Y_train, Y_test = train_test_split(
        X, Y,
        test_size=0.2,      # 80% train, 20% test
        random_state=SEED,
        stratify=Y          # Stratify based on the target variable
    )
    print(f"Training set size: {len(X_train)}")
    print(f"Testing set size: {len(X_test)}")
    print("\nTraining set label distribution:")
    print(Y_train.value_counts(normalize=True))
    print("\nTesting set label distribution:")
    print(Y_test.value_counts(normalize=True))
else:
    print("Cannot prepare data as df_processed is not available.")

```

```

# exit() # Stop if data isn't ready

# --- Model 1: TF-IDF + Logistic Regression ---

if 'X_train' in locals(): # Check if data split was successful
    print("\n--- Training Model 1: TF-IDF + Logistic Regression ---")

    # 1. TF-IDF Vectorization
    print("Applying TF-IDF Vectorizer...")
    start_time = time.time()
    vectorizer = TfidfVectorizer(max_features=10000, ngram_range=(1, 2)) # Increased features, added bigrams
    X_train_tfidf = vectorizer.fit_transform(X_train)
    X_test_tfidf = vectorizer.transform(X_test)
    print(f"TF-IDF fitting/transformation done in {time.time() - start_time:.2f} seconds.")
    print(f"TF-IDF Matrix Shape (Train): {X_train_tfidf.shape}")

    # 2. Logistic Regression Training
    print("Training Logistic Regression model...")
    start_time = time.time()
    # Increased max_iter, consider trying 'saga' solver for large datasets if 'liblinear' is slow
    log_reg_model = LogisticRegression(
        solver='liblinear', # Good for high-dimensional sparse data
        random_state=SEED,
        max_iter=1000,      # Increase max iterations
        C=1.0               # Regularization strength (default)
    )
    log_reg_model.fit(X_train_tfidf, Y_train)
    print(f"Logistic Regression training done in {time.time() - start_time:.2f} seconds.")

    # 3. Evaluation
    print("\nEvaluating Logistic Regression model...")
    Y_pred_log_reg = log_reg_model.predict(X_test_tfidf)

    print("\nLogistic Regression Classification Report:")
    target_names = ['Negative (1-2 Stars)', 'Neutral (3 Stars)', 'Positive (4-5 Stars)']
    print(classification_report(Y_test, Y_pred_log_reg, target_names=target_names, digits=4))

    # Optional: Confusion Matrix Plot
    print("Generating Confusion Matrix for Logistic Regression...")
    try:
        cm_log_reg = confusion_matrix(Y_test, Y_pred_log_reg)
        disp_log_reg = ConfusionMatrixDisplay(confusion_matrix=cm_log_reg, display_labels=target_names)
        fig, ax = plt.subplots(figsize=(8, 6))
        disp_log_reg.plot(cmap=plt.cm.Blues, ax=ax, xticks_rotation='vertical')
        plt.title('Logistic Regression Confusion Matrix (Stars)')
        plt.tight_layout()
        plt.savefig(os.path.join(output_dir, 'log_reg_confusion_matrix_stars.png'))
        # plt.show() # Uncomment to display plot directly
        plt.close(fig) # Close the figure to prevent displaying in console output if not desired
        print(f"Confusion matrix saved to {os.path.join(output_dir, 'log_reg_confusion_matrix_stars.png')}")
    except Exception as e:
        print(f"Could not generate/save confusion matrix plot: {e}")

else:
    print("Skipping TF-IDF + Logistic Regression model training as data is not prepared.")

# --- Model 2: Transformer (DistilBERT) ---

if 'X_train' in locals(): # Check if data split was successful
    print("\n--- Preparing for Model 2: Transformer (DistilBERT) ---")

    # 1. Tokenization
    MODEL_NAME = 'distilbert-base-uncased'
    print(f>Loading tokenizer: {MODEL_NAME}...")
    try:
        tokenizer = DistilBertTokenizer.from_pretrained(MODEL_NAME)
    except Exception as e:
        print(f>Error loading tokenizer: {e}. Cannot proceed with Transformer.")
        tokenizer = None # Set tokenizer to None to prevent further errors

    if tokenizer:
        print("Tokenizing data...")
        start_time = time.time()
        # Convert pandas Series to lists for tokenizer
        train_texts = X_train.tolist()
        test_texts = X_test.tolist()

```

```

# Tokenize - Use appropriate max_length
MAX_LEN = 256 # Adjust based on review length analysis and memory constraints
train_encodings = tokenizer(train_texts, truncation=True, padding=True, max_length=MAX_LEN)
test_encodings = tokenizer(test_texts, truncation=True, padding=True, max_length=MAX_LEN)
print(f"Tokenization done in {time.time() - start_time:.2f} seconds.")

# Clear text lists to save memory
del train_texts, test_texts
gc.collect()

# 2. Create PyTorch Datasets
class SentimentDataset(Dataset):
    def __init__(self, encodings, labels):
        self.encodings = encodings
        self.labels = labels

    def __getitem__(self, idx):
        # Correctly handle accessing dictionary items
        item = {key: torch.tensor(val[idx]) for key, val in self.encodings.items()}
        # Labels should be LongTensor for CrossEntropyLoss
        item['labels'] = torch.tensor(self.labels[idx], dtype=torch.long)
        return item

    def __len__(self):
        # Use 'input_ids' length which is standard
        return len(self.encodings['input_ids'])

# Convert labels (pandas Series) to numpy arrays BEFORE creating dataset
Y_train_np = Y_train.values
Y_test_np = Y_test.values

train_dataset = SentimentDataset(train_encodings, Y_train_np)
test_dataset = SentimentDataset(test_encodings, Y_test_np)
print("PyTorch datasets created.")

# 3. Define Model
print(f"Loading pre-trained model: {MODEL_NAME}...")
try:
    # Load for sequence classification with 3 labels (neg, neu, pos)
    transformer_model = DistilBertForSequenceClassification.from_pretrained(
        MODEL_NAME,
        num_labels=3 # Directly specify the number of output classes
    )
    transformer_model.to(device) # Move model to GPU/CPU
    print("Transformer model loaded and moved to device.")
except Exception as e:
    print(f"Error loading Transformer model: {e}")
    transformer_model = None # Set model to None

# 4. Training Setup (only if model loaded successfully)
if transformer_model:
    print("Setting up training parameters...")
    # Hyperparameters (adjust as needed)
    LEARNING_RATE = 5e-5
    EPOCHS = 1 # Start with 1 epoch for faster iteration, increase later
    BATCH_SIZE = 16 # Adjust based on GPU memory (16 or 32 are common)

    # Dataloaders
    train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE, shuffle=True)
    test_loader = DataLoader(test_dataset, batch_size=BATCH_SIZE * 2) # Larger batch size for eval is ok

    # Optimizer and Scheduler
    optimizer = AdamW(transformer_model.parameters(), lr=LEARNING_RATE)
    num_training_steps = EPOCHS * len(train_loader)
    lr_scheduler = get_linear_schedule_with_warmup(
        optimizer,
        num_warmup_steps=0, # You can add warmup steps if desired
        num_training_steps=num_training_steps
    )

    # Loss Function
    loss_fn = torch.nn.CrossEntropyLoss()
    print("Training setup complete.")

# 5. Training Loop

```

```

print("\n--- Training Transformer Model ---")
transformer_model.train() # Set model to training mode

for epoch in range(EPOCHS):
    print(f"\n--- Epoch {epoch + 1}/{EPOCHS} ---")
    epoch_start_time = time.time()
    total_loss = 0

    for i, batch in enumerate(train_loader):
        optimizer.zero_grad()

        # Move batch to device
        input_ids = batch['input_ids'].to(device)
        attention_mask = batch['attention_mask'].to(device)
        labels = batch['labels'].to(device)

        # Forward pass
        outputs = transformer_model(
            input_ids=input_ids,
            attention_mask=attention_mask,
            labels=labels # Pass labels for loss calculation within the model
        )

        loss = outputs.loss # Model calculates loss when labels are provided
        total_loss += loss.item()

        # Backward pass and optimization
        loss.backward()
        torch.nn.utils.clip_grad_norm_(transformer_model.parameters(), 1.0) # Gradient clipping
        optimizer.step()
        lr_scheduler.step()

        if (i + 1) % 100 == 0: # Print progress every 100 batches
            print(f"  Batch {i+1}/{len(train_loader)}, Loss: {loss.item():.4f}")

    avg_train_loss = total_loss / len(train_loader)
    epoch_time = time.time() - epoch_start_time
    print(f"Epoch {epoch + 1} completed in {epoch_time:.2f}s. Average Training Loss: {avg_train_loss:.4f}")

print("Transformer training finished.")

# Optional: Save the fine-tuned model
print(f"Saving fine-tuned Transformer model to {output_dir}...")
try:
    transformer_model.save_pretrained(output_dir)
    tokenizer.save_pretrained(output_dir)
    print("Model and tokenizer saved successfully.")
except Exception as e:
    print(f"Error saving model/tokenizer: {e}")

# 6. Evaluation
print("\nEvaluating Transformer model...")
transformer_model.eval() # Set model to evaluation mode
all_preds_transformer = []
all_labels_transformer = []

with torch.no_grad(): # Disable gradient calculations
    for batch in test_loader:
        input_ids = batch['input_ids'].to(device)
        attention_mask = batch['attention_mask'].to(device)
        labels = batch['labels'].to(device)

        outputs = transformer_model(input_ids=input_ids, attention_mask=attention_mask)
        logits = outputs.logits
        predictions = torch.argmax(logits, dim=-1)

        all_preds_transformer.extend(predictions.cpu().numpy())
        all_labels_transformer.extend(labels.cpu().numpy())

print("\nTransformer Classification Report (Stars):")
print(classification_report(all_labels_transformer, all_preds_transformer, target_names=target_names, digits=4))

# Optional: Confusion Matrix Plot
print("Generating Confusion Matrix for Transformer...")
try:
    cm_transformer = confusion_matrix(all_labels_transformer, all_preds_transformer)
    fig_transformer = ConfusionMatrixDisplay(confusion_matrix=cm_transformer, display_labels=target_names)

```

```

disp_transformer = ConfusionMatrixDisplay(confusion_matrix_cm_transformer, display_labels=largest_names)
fig, ax = plt.subplots(figsize=(8, 6))
disp_transformer.plot(cmap=plt.cm.Blues, ax=ax, xticks_rotation='vertical')
plt.title('Transformer Confusion Matrix (Stars)')
plt.tight_layout()
plt.savefig(os.path.join(output_dir, 'transformer_confusion_matrix_stars.png'))
# plt.show() # Uncomment to display plot directly
plt.close(fig) # Close the figure
print(f"Confusion matrix saved to {os.path.join(output_dir, 'transformer_confusion_matrix_stars.png')}")
except Exception as e:
    print(f"Could not generate/save confusion matrix plot: {e}")

else:
    print("Skipping Transformer training and evaluation due to model loading error.")
else:
    print("Skipping Transformer training and evaluation due to tokenizer loading error.")

else:
    print("Skipping Model Training and Evaluation as data is not prepared.")

print("\n--- Script Finished ---")

```



```

Requirement already satisfied: nltk in /usr/local/lib/python3.11/dist-packages (3.9.1)
Requirement already satisfied: click in /usr/local/lib/python3.11/dist-packages (from nltk) (8.1.8)
Requirement already satisfied: joblib in /usr/local/lib/python3.11/dist-packages (from nltk) (1.4.2)
Requirement already satisfied: regex<=2021.8.3 in /usr/local/lib/python3.11/dist-packages (from nltk) (2024.11.6)
Requirement already satisfied: tqdm in /usr/local/lib/python3.11/dist-packages (from nltk) (4.67.1)
Downloading NLTK package: wordnet...
Downloading NLTK package: punkt_tab...
NLTK data check complete.
Using device: cuda
Downloading NLTK package: wordnet...
NLTK data check complete.
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
Running in Colab environment.
Model output directory: /content/drive/My Drive/Colab Notebooks/SentimentModels_Stars

Loading review data from: /content/drive/My Drive/Colab Notebooks/SentimentModels_Stars/yelp_academic_dataset_review.json
Review data loaded in 103.98 seconds. Shape: (6990280, 5)

Loading business data from: /content/drive/My Drive/Colab Notebooks/SentimentModels_Stars/yelp_academic_dataset_business.json
Business data loaded in 2.25 seconds. Shape: (150346, 2)

Found 64629 unique target restaurant business IDs.
Reviews filtered by date (2019-01-01 to 2024-12-31). Shape: (2111695, 5)
Reviews filtered by target business IDs. Shape: (1525330, 5)

Final shape for processing: (1525330, 5)
      review_id \
194087  F6VdYuJiefNBfn3HNELv0A
194089  nAMDCKE1SKxOhzm9Lpt6Eg
194093  3CmdoGKBZUX3Nb5IfbztMg
203214  -4Nv_JAolCM0gzKM4DZpmQ
204271  uObbDRxP_cwJVADckZJDZw

      text  stars
194087  The food is INCREDIBLE! We didn't have time to...      5
194089  We had a great time, and excellent service. All...      5
194093  My favorite coffee shop in New Orleans for sur...      5
203214  Old school circa 1979, the cozy, intimate banq...      4
204271  Delicious! Tried this place on Saturday and it...      5

Intermediate dataframes cleaned up.

Starting text preprocessing...
Text preprocessing completed in 742.47 seconds.

Original vs Processed Text Examples:
--- Example 1 ---
Original: The food is INCREDIBLE! We didn't have time to get any of the cocktails, but they had some pretty creative and classy ones on
Processed: food incredible didnt time get cocktail pretty creative classy one menu going back alone anyways way taking unique twist thin
-----
--- Example 2 ---
Original: We had a great time, and excellent service. All food items were fresh, unique, and high quality. Nothing appeared to be pre-ma
Except for the ice cream which was exceptionally good. The waitress wa...
Processed: great time excellent service food item fresh unique high quality nothing appeared premade except ice cream exceptionally good
-----
--- Example 3 ---
Original: My favorite coffee shop in New Orleans for sure! Perfect to study (quiet environment), super nice and hospital staff, such a b
Processed: favorite coffee shop new orleans sure perfect study quiet environment super nice hospital staff bright space coffee full flav
-----

Removed 9 rows with empty processed text.
Shape after cleaning empty processed texts: (1525321, 4)

Mapping star ratings to sentiment labels...
Value counts for 'overall_sentiment_stars':
overall_sentiment_stars
2    1056262
0     335931
1     133128
Name: count, dtype: int64
Shape after mapping stars: (1525321, 5)

Splitting data into training and testing sets...
Training set size: 1220256
Testing set size: 305065

Training set label distribution:
overall_sentiment_stars
2    0.692485
0    0.220237
1    0.087278
Name: proportion, dtype: float64

Testing set label distribution:

```

```
overall_sentiment_stars
```

```
2    0.692485
```

```
0    0.220235
```

```
1    0.087280
```

```
Name: proportion, dtype: float64
```

```
--- Training Model 1: TF-IDF + Logistic Regression ---
```

```
Applying TF-IDF Vectorizer...
```

```
TF-IDF fitting/transformation done in 168.55 seconds.
```

```
TF-IDF Matrix Shape (Train): (1220256, 10000)
```

```
Training Logistic Regression model...
```

```
Logistic Regression training done in 85.68 seconds.
```

```
Evaluating Logistic Regression model...
```

```
Logistic Regression Classification Report:
```

	precision	recall	f1-score	support
Negative (1-2 Stars)	0.8421	0.8979	0.8691	67186
Neutral (3 Stars)	0.5858	0.2682	0.3679	26626
Positive (4-5 Stars)	0.9317	0.9757	0.9532	211253
accuracy			0.8968	305065
macro avg	0.7865	0.7139	0.7301	305065
weighted avg	0.8818	0.8968	0.8836	305065

```
Generating Confusion Matrix for Logistic Regression...
```

```
Confusion matrix saved to /content/drive/My Drive/Colab Notebooks/SentimentModels_Stars/log_reg_confusion_matrix_stars.png
```

```
--- Preparing for Model 2: Transformer (DistilBERT) ---
```

```
Loading tokenizer: distilbert-base-uncased...
```

```
Tokenizing data...
```

```
Tokenization done in 1431.46 seconds.
```

```
PyTorch datasets created.
```

```
Loading pre-trained model: distilbert-base-uncased...
```

```
Xet Storage is enabled for this repo, but the 'hf_xet' package is not installed. Falling back to regular HTTP download. For better perf
WARNING:huggingface_hub.file_download:Xet Storage is enabled for this repo, but the 'hf_xet' package is not installed. Falling back to r
model.safetensors: 100% 268M/268M [00:01<00:00, 243MB/s]
```

```
Some weights of DistilBertForSequenceClassification were not initialized from the model checkpoint at distilbert-base-uncased and are ne
You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.
```

```
Transformer model loaded and moved to device.
```

```
Setting up training parameters...
```

```
Training setup complete.
```

```
--- Training Transformer Model ---
```

```
--- Epoch 1/1 ---
```

```
Batch 100/76266, Loss: 0.5340
```

```
Batch 200/76266, Loss: 0.6773
```

```
Batch 300/76266, Loss: 0.2966
```

```
Batch 400/76266, Loss: 0.4500
```

```
Batch 500/76266, Loss: 0.4007
```

```
Batch 600/76266, Loss: 0.4156
```

```
Batch 700/76266, Loss: 0.4678
```

```
Batch 800/76266, Loss: 0.1845
```

```
Batch 900/76266, Loss: 0.2397
```

```
Batch 1000/76266, Loss: 0.6178
```

```
Batch 1100/76266, Loss: 0.7052
```

```
Batch 1200/76266, Loss: 0.3394
```

```
Batch 1300/76266, Loss: 0.2094
```

```
Batch 1400/76266, Loss: 0.2284
```

```
Batch 1500/76266, Loss: 0.1981
```

```
Batch 1600/76266, Loss: 0.6430
```

```
Batch 1700/76266, Loss: 0.1424
```

```
Batch 1800/76266, Loss: 0.6443
```

```
Batch 1900/76266, Loss: 0.2880
```

```
Batch 2000/76266, Loss: 0.1551
```

```
Batch 2100/76266, Loss: 0.2638
```

```
Batch 2200/76266, Loss: 0.6675
```

```
Batch 2300/76266, Loss: 0.1724
```

```
Batch 2400/76266, Loss: 0.3672
```

```
Batch 2500/76266, Loss: 0.2051
```

```
Batch 2600/76266, Loss: 0.4442
```

```
Batch 2700/76266, Loss: 0.2209
```

```
Batch 2800/76266, Loss: 0.2179
```

```
Batch 2900/76266, Loss: 0.3747
```

```
Batch 3000/76266, Loss: 0.4530
```

```
Batch 3100/76266, Loss: 0.4836
```

```
Batch 3200/76266, Loss: 0.0878
```

```
Batch 3300/76266, Loss: 0.0563
```

```
Batch 3400/76266, Loss: 0.8678
```

```
Batch 3500/76266, Loss: 0.0509
```

```
Batch 3600/76266, Loss: 0.2083
```

```
Batch 3700/76266, Loss: 0.1087
```

Batch 3800/76266, Loss: 0.3977
Batch 3900/76266, Loss: 0.0388
Batch 4000/76266, Loss: 0.2475
Batch 4100/76266, Loss: 0.3224
Batch 4200/76266, Loss: 0.2589
Batch 4300/76266, Loss: 0.4166
Batch 4400/76266, Loss: 0.4341
Batch 4500/76266, Loss: 0.6347
Batch 4600/76266, Loss: 0.1714
Batch 4700/76266, Loss: 0.4440
Batch 4800/76266, Loss: 0.2793
Batch 4900/76266, Loss: 0.1864
Batch 5000/76266, Loss: 0.0904
Batch 5100/76266, Loss: 0.3468
Batch 5200/76266, Loss: 0.4910
Batch 5300/76266, Loss: 0.3480
Batch 5400/76266, Loss: 0.1406
Batch 5500/76266, Loss: 0.0688
Batch 5600/76266, Loss: 0.3523
Batch 5700/76266, Loss: 0.2756
Batch 5800/76266, Loss: 0.1904
Batch 5900/76266, Loss: 0.4647
Batch 6000/76266, Loss: 0.3535
Batch 6100/76266, Loss: 0.7530
Batch 6200/76266, Loss: 0.3152
Batch 6300/76266, Loss: 0.4585
Batch 6400/76266, Loss: 0.0644
Batch 6500/76266, Loss: 0.5141
Batch 6600/76266, Loss: 0.3778
Batch 6700/76266, Loss: 0.1358
Batch 6800/76266, Loss: 0.8917
Batch 6900/76266, Loss: 0.4081
Batch 7000/76266, Loss: 0.4190
Batch 7100/76266, Loss: 0.0915
Batch 7200/76266, Loss: 0.3736
Batch 7300/76266, Loss: 0.1572
Batch 7400/76266, Loss: 0.5667
Batch 7500/76266, Loss: 0.1196
Batch 7600/76266, Loss: 0.2421
Batch 7700/76266, Loss: 0.0785
Batch 7800/76266, Loss: 0.4006
Batch 7900/76266, Loss: 0.3990
Batch 8000/76266, Loss: 0.1847
Batch 8100/76266, Loss: 0.4378
Batch 8200/76266, Loss: 0.0314
Batch 8300/76266, Loss: 0.1833
Batch 8400/76266, Loss: 0.0142
Batch 8500/76266, Loss: 0.3637
Batch 8600/76266, Loss: 0.3589
Batch 8700/76266, Loss: 0.3494
Batch 8800/76266, Loss: 0.1835
Batch 8900/76266, Loss: 0.5619
Batch 9000/76266, Loss: 0.1448
Batch 9100/76266, Loss: 0.0747
Batch 9200/76266, Loss: 0.3460
Batch 9300/76266, Loss: 0.1302
Batch 9400/76266, Loss: 0.5252
Batch 9500/76266, Loss: 0.2241
Batch 9600/76266, Loss: 0.4040
Batch 9700/76266, Loss: 0.7120
Batch 9800/76266, Loss: 0.2774
Batch 9900/76266, Loss: 0.1334
Batch 10000/76266, Loss: 0.0771
Batch 10100/76266, Loss: 0.3142
Batch 10200/76266, Loss: 0.3135
Batch 10300/76266, Loss: 0.2847
Batch 10400/76266, Loss: 0.1504
Batch 10500/76266, Loss: 0.1729
Batch 10600/76266, Loss: 0.2889
Batch 10700/76266, Loss: 0.3303
Batch 10800/76266, Loss: 0.2106
Batch 10900/76266, Loss: 0.1800
Batch 11000/76266, Loss: 0.1549
Batch 11100/76266, Loss: 0.0362
Batch 11200/76266, Loss: 0.3650
Batch 11300/76266, Loss: 0.0741
Batch 11400/76266, Loss: 0.9303
Batch 11500/76266, Loss: 0.1795
Batch 11600/76266, Loss: 0.2066
Batch 11700/76266, Loss: 0.3615
Batch 11800/76266, Loss: 0.1145
Batch 11900/76266, Loss: 0.1314
Batch 12000/76266, Loss: 0.8653
Batch 12100/76266, Loss: 0.1798
Batch 12200/76266, Loss: 0.1985

Batch 12300/76266, Loss: 0.1465
Batch 12400/76266, Loss: 0.0344
Batch 12500/76266, Loss: 0.4596
Batch 12600/76266, Loss: 0.4708
Batch 12700/76266, Loss: 0.5680
Batch 12800/76266, Loss: 0.2572
Batch 12900/76266, Loss: 0.2293
Batch 13000/76266, Loss: 0.6830
Batch 13100/76266, Loss: 0.2027
Batch 13200/76266, Loss: 0.3955
Batch 13300/76266, Loss: 0.0403
Batch 13400/76266, Loss: 0.2973
Batch 13500/76266, Loss: 0.1098
Batch 13600/76266, Loss: 0.3485
Batch 13700/76266, Loss: 0.2821
Batch 13800/76266, Loss: 0.0181
Batch 13900/76266, Loss: 0.0960
Batch 14000/76266, Loss: 0.4831
Batch 14100/76266, Loss: 0.2953
Batch 14200/76266, Loss: 0.5710
Batch 14300/76266, Loss: 0.6983
Batch 14400/76266, Loss: 0.1379
Batch 14500/76266, Loss: 0.1261
Batch 14600/76266, Loss: 0.3587
Batch 14700/76266, Loss: 0.1788
Batch 14800/76266, Loss: 0.1887
Batch 14900/76266, Loss: 0.6215
Batch 15000/76266, Loss: 0.3909
Batch 15100/76266, Loss: 0.6288
Batch 15200/76266, Loss: 0.2005
Batch 15300/76266, Loss: 0.3478
Batch 15400/76266, Loss: 0.2648
Batch 15500/76266, Loss: 0.3259
Batch 15600/76266, Loss: 0.2630
Batch 15700/76266, Loss: 0.2173
Batch 15800/76266, Loss: 0.1709
Batch 15900/76266, Loss: 0.2110
Batch 16000/76266, Loss: 0.5104
Batch 16100/76266, Loss: 0.2958
Batch 16200/76266, Loss: 0.2455
Batch 16300/76266, Loss: 0.2153
Batch 16400/76266, Loss: 0.1151
Batch 16500/76266, Loss: 0.5756
Batch 16600/76266, Loss: 0.1972
Batch 16700/76266, Loss: 0.0900
Batch 16800/76266, Loss: 0.0517
Batch 16900/76266, Loss: 0.2973
Batch 17000/76266, Loss: 0.4213
Batch 17100/76266, Loss: 0.3635
Batch 17200/76266, Loss: 0.8241
Batch 17300/76266, Loss: 0.3065
Batch 17400/76266, Loss: 0.1346
Batch 17500/76266, Loss: 0.0776
Batch 17600/76266, Loss: 0.1636
Batch 17700/76266, Loss: 0.5242
Batch 17800/76266, Loss: 0.2925
Batch 17900/76266, Loss: 0.0741
Batch 18000/76266, Loss: 0.1500
Batch 18100/76266, Loss: 0.2335
Batch 18200/76266, Loss: 0.1509
Batch 18300/76266, Loss: 0.1912
Batch 18400/76266, Loss: 0.6667
Batch 18500/76266, Loss: 0.0620
Batch 18600/76266, Loss: 0.1056
Batch 18700/76266, Loss: 0.2264
Batch 18800/76266, Loss: 0.3236
Batch 18900/76266, Loss: 0.3941
Batch 19000/76266, Loss: 0.2868
Batch 19100/76266, Loss: 0.0379
Batch 19200/76266, Loss: 0.9436
Batch 19300/76266, Loss: 0.2272
Batch 19400/76266, Loss: 0.5583
Batch 19500/76266, Loss: 0.3726
Batch 19600/76266, Loss: 0.2585
Batch 19700/76266, Loss: 0.5530
Batch 19800/76266, Loss: 0.3000
Batch 19900/76266, Loss: 0.4032
Batch 20000/76266, Loss: 0.0590
Batch 20100/76266, Loss: 0.2016
Batch 20200/76266, Loss: 0.0824
Batch 20300/76266, Loss: 0.2599
Batch 20400/76266, Loss: 0.2412
Batch 20500/76266, Loss: 0.6757
Batch 20600/76266, Loss: 0.0588
Batch 20700/76266, Loss: 0.1451

Batch 20800/76266, Loss: 0.1403
Batch 20900/76266, Loss: 0.3298
Batch 21000/76266, Loss: 0.2396
Batch 21100/76266, Loss: 0.1119
Batch 21200/76266, Loss: 0.3538
Batch 21300/76266, Loss: 0.5660
Batch 21400/76266, Loss: 0.1805
Batch 21500/76266, Loss: 0.1307
Batch 21600/76266, Loss: 0.5533
Batch 21700/76266, Loss: 0.2675
Batch 21800/76266, Loss: 0.3058
Batch 21900/76266, Loss: 0.0457
Batch 22000/76266, Loss: 0.3035
Batch 22100/76266, Loss: 0.2083
Batch 22200/76266, Loss: 0.0758
Batch 22300/76266, Loss: 0.1037
Batch 22400/76266, Loss: 1.2371
Batch 22500/76266, Loss: 0.1696
Batch 22600/76266, Loss: 0.4264
Batch 22700/76266, Loss: 0.3436
Batch 22800/76266, Loss: 0.2876
Batch 22900/76266, Loss: 0.1902
Batch 23000/76266, Loss: 0.0879
Batch 23100/76266, Loss: 0.1038
Batch 23200/76266, Loss: 0.4162
Batch 23300/76266, Loss: 0.2886
Batch 23400/76266, Loss: 0.1517
Batch 23500/76266, Loss: 0.0430
Batch 23600/76266, Loss: 0.1680
Batch 23700/76266, Loss: 0.1221
Batch 23800/76266, Loss: 0.0240
Batch 23900/76266, Loss: 0.0273
Batch 24000/76266, Loss: 0.2004
Batch 24100/76266, Loss: 0.4054
Batch 24200/76266, Loss: 0.2407
Batch 24300/76266, Loss: 0.3444
Batch 24400/76266, Loss: 0.4321
Batch 24500/76266, Loss: 0.1519
Batch 24600/76266, Loss: 0.3836
Batch 24700/76266, Loss: 0.3534
Batch 24800/76266, Loss: 0.4114
Batch 24900/76266, Loss: 0.4499
Batch 25000/76266, Loss: 0.2583
Batch 25100/76266, Loss: 0.0159
Batch 25200/76266, Loss: 0.0915
Batch 25300/76266, Loss: 0.0579
Batch 25400/76266, Loss: 0.2026
Batch 25500/76266, Loss: 0.2464
Batch 25600/76266, Loss: 0.1053
Batch 25700/76266, Loss: 0.1308
Batch 25800/76266, Loss: 0.1505
Batch 25900/76266, Loss: 0.2942
Batch 26000/76266, Loss: 0.1253
Batch 26100/76266, Loss: 0.2011
Batch 26200/76266, Loss: 0.0727
Batch 26300/76266, Loss: 0.2535
Batch 26400/76266, Loss: 0.2839
Batch 26500/76266, Loss: 0.3673
Batch 26600/76266, Loss: 0.1141
Batch 26700/76266, Loss: 0.2099
Batch 26800/76266, Loss: 0.3209
Batch 26900/76266, Loss: 0.1909
Batch 27000/76266, Loss: 0.3985
Batch 27100/76266, Loss: 0.2284
Batch 27200/76266, Loss: 0.1544
Batch 27300/76266, Loss: 0.2097
Batch 27400/76266, Loss: 0.1359
Batch 27500/76266, Loss: 0.6108
Batch 27600/76266, Loss: 0.0288
Batch 27700/76266, Loss: 0.4162
Batch 27800/76266, Loss: 0.2017
Batch 27900/76266, Loss: 0.2273
Batch 28000/76266, Loss: 0.2025
Batch 28100/76266, Loss: 0.2505
Batch 28200/76266, Loss: 0.5226
Batch 28300/76266, Loss: 0.0927
Batch 28400/76266, Loss: 0.7566
Batch 28500/76266, Loss: 0.7468
Batch 28600/76266, Loss: 0.1568
Batch 28700/76266, Loss: 0.3711
Batch 28800/76266, Loss: 0.1064
Batch 28900/76266, Loss: 0.1345
Batch 29000/76266, Loss: 0.1409
Batch 29100/76266, Loss: 0.1630
Batch 29200/76266, Loss: 0.1201

Batch 29300/76266, Loss: 0.4208
Batch 29400/76266, Loss: 0.1758
Batch 29500/76266, Loss: 0.1908
Batch 29600/76266, Loss: 0.6850
Batch 29700/76266, Loss: 0.1700
Batch 29800/76266, Loss: 0.4893
Batch 29900/76266, Loss: 0.1800
Batch 30000/76266, Loss: 0.0940
Batch 30100/76266, Loss: 0.0285
Batch 30200/76266, Loss: 0.1182
Batch 30300/76266, Loss: 0.3940
Batch 30400/76266, Loss: 0.4766
Batch 30500/76266, Loss: 0.1946
Batch 30600/76266, Loss: 0.2699
Batch 30700/76266, Loss: 0.3319
Batch 30800/76266, Loss: 0.7206
Batch 30900/76266, Loss: 0.1906
Batch 31000/76266, Loss: 0.4058
Batch 31100/76266, Loss: 0.5036
Batch 31200/76266, Loss: 0.0383
Batch 31300/76266, Loss: 0.6663
Batch 31400/76266, Loss: 0.4023
Batch 31500/76266, Loss: 0.2322
Batch 31600/76266, Loss: 0.2445
Batch 31700/76266, Loss: 0.3622
Batch 31800/76266, Loss: 0.1808
Batch 31900/76266, Loss: 0.2212
Batch 32000/76266, Loss: 0.2234
Batch 32100/76266, Loss: 0.1567
Batch 32200/76266, Loss: 0.0293
Batch 32300/76266, Loss: 0.5477
Batch 32400/76266, Loss: 0.4529
Batch 32500/76266, Loss: 0.0697
Batch 32600/76266, Loss: 0.1494
Batch 32700/76266, Loss: 0.1911
Batch 32800/76266, Loss: 0.1861
Batch 32900/76266, Loss: 0.4723
Batch 33000/76266, Loss: 0.3417
Batch 33100/76266, Loss: 0.5177
Batch 33200/76266, Loss: 0.2482
Batch 33300/76266, Loss: 0.2124
Batch 33400/76266, Loss: 0.2112
Batch 33500/76266, Loss: 0.3821
Batch 33600/76266, Loss: 0.3847
Batch 33700/76266, Loss: 0.7499
Batch 33800/76266, Loss: 0.2162
Batch 33900/76266, Loss: 0.1748
Batch 34000/76266, Loss: 0.0527
Batch 34100/76266, Loss: 0.1489
Batch 34200/76266, Loss: 0.5333
Batch 34300/76266, Loss: 0.3116
Batch 34400/76266, Loss: 0.4165
Batch 34500/76266, Loss: 0.1053
Batch 34600/76266, Loss: 0.0705
Batch 34700/76266, Loss: 0.2257
Batch 34800/76266, Loss: 0.9862
Batch 34900/76266, Loss: 0.7517
Batch 35000/76266, Loss: 0.3208
Batch 35100/76266, Loss: 0.1876
Batch 35200/76266, Loss: 0.1213
Batch 35300/76266, Loss: 0.3522
Batch 35400/76266, Loss: 0.1091
Batch 35500/76266, Loss: 0.0563
Batch 35600/76266, Loss: 0.2302
Batch 35700/76266, Loss: 0.0158
Batch 35800/76266, Loss: 0.0889
Batch 35900/76266, Loss: 0.2125
Batch 36000/76266, Loss: 0.4324
Batch 36100/76266, Loss: 0.0470
Batch 36200/76266, Loss: 0.1559
Batch 36300/76266, Loss: 0.3926
Batch 36400/76266, Loss: 0.0192
Batch 36500/76266, Loss: 0.1008
Batch 36600/76266, Loss: 0.2111
Batch 36700/76266, Loss: 0.0805
Batch 36800/76266, Loss: 0.0803
Batch 36900/76266, Loss: 0.7228
Batch 37000/76266, Loss: 0.0200
Batch 37100/76266, Loss: 0.4083
Batch 37200/76266, Loss: 0.1512
Batch 37300/76266, Loss: 0.1489
Batch 37400/76266, Loss: 0.1402
Batch 37500/76266, Loss: 0.0389
Batch 37600/76266, Loss: 0.1648

Batch 37700/76266, Loss: 0.1894
Batch 37800/76266, Loss: 0.2594
Batch 37900/76266, Loss: 0.6321
Batch 38000/76266, Loss: 0.2561
Batch 38100/76266, Loss: 0.2040
Batch 38200/76266, Loss: 0.2495
Batch 38300/76266, Loss: 0.4147
Batch 38400/76266, Loss: 0.4345
Batch 38500/76266, Loss: 0.5344
Batch 38600/76266, Loss: 0.1857
Batch 38700/76266, Loss: 0.0917
Batch 38800/76266, Loss: 0.2061
Batch 38900/76266, Loss: 0.7365
Batch 39000/76266, Loss: 0.3640
Batch 39100/76266, Loss: 0.0251
Batch 39200/76266, Loss: 0.1096
Batch 39300/76266, Loss: 0.0503
Batch 39400/76266, Loss: 0.3138
Batch 39500/76266, Loss: 0.0401
Batch 39600/76266, Loss: 0.2115
Batch 39700/76266, Loss: 0.2448
Batch 39800/76266, Loss: 0.1817
Batch 39900/76266, Loss: 0.0813
Batch 40000/76266, Loss: 0.6714
Batch 40100/76266, Loss: 0.3466
Batch 40200/76266, Loss: 0.1211
Batch 40300/76266, Loss: 0.2793
Batch 40400/76266, Loss: 0.5252
Batch 40500/76266, Loss: 0.0350
Batch 40600/76266, Loss: 0.1226
Batch 40700/76266, Loss: 0.1659
Batch 40800/76266, Loss: 0.0135
Batch 40900/76266, Loss: 0.2563
Batch 41000/76266, Loss: 0.5858
Batch 41100/76266, Loss: 0.5644
Batch 41200/76266, Loss: 0.3902
Batch 41300/76266, Loss: 0.6350
Batch 41400/76266, Loss: 0.2300
Batch 41500/76266, Loss: 0.0526
Batch 41600/76266, Loss: 0.3021
Batch 41700/76266, Loss: 0.2823
Batch 41800/76266, Loss: 0.4270
Batch 41900/76266, Loss: 0.2933
Batch 42000/76266, Loss: 0.2391
Batch 42100/76266, Loss: 0.0614
Batch 42200/76266, Loss: 0.0950
Batch 42300/76266, Loss: 0.3403
Batch 42400/76266, Loss: 0.5573
Batch 42500/76266, Loss: 0.1448
Batch 42600/76266, Loss: 0.0829
Batch 42700/76266, Loss: 0.0565
Batch 42800/76266, Loss: 0.6295
Batch 42900/76266, Loss: 0.2046
Batch 43000/76266, Loss: 0.2257
Batch 43100/76266, Loss: 0.0887
Batch 43200/76266, Loss: 0.4705
Batch 43300/76266, Loss: 0.0933
Batch 43400/76266, Loss: 0.2845
Batch 43500/76266, Loss: 0.1276
Batch 43600/76266, Loss: 0.2431
Batch 43700/76266, Loss: 0.0943
Batch 43800/76266, Loss: 0.2480
Batch 43900/76266, Loss: 0.1250
Batch 44000/76266, Loss: 0.0570
Batch 44100/76266, Loss: 0.4404
Batch 44200/76266, Loss: 0.0509
Batch 44300/76266, Loss: 0.4571
Batch 44400/76266, Loss: 0.1884
Batch 44500/76266, Loss: 0.1807
Batch 44600/76266, Loss: 0.0016
Batch 44700/76266, Loss: 0.2233
Batch 44800/76266, Loss: 0.1427
Batch 44900/76266, Loss: 0.5015
Batch 45000/76266, Loss: 0.0083
Batch 45100/76266, Loss: 0.1748
Batch 45200/76266, Loss: 0.3983
Batch 45300/76266, Loss: 0.3764
Batch 45400/76266, Loss: 0.3097
Batch 45500/76266, Loss: 0.2904
Batch 45600/76266, Loss: 0.2128
Batch 45700/76266, Loss: 0.0451
Batch 45800/76266, Loss: 0.5224
Batch 45900/76266, Loss: 0.5690
Batch 46000/76266, Loss: 0.3716
Batch 46100/76266, Loss: 0.0599

Batch 46200/76266, Loss: 0.4531
Batch 46300/76266, Loss: 0.4114
Batch 46400/76266, Loss: 0.2170
Batch 46500/76266, Loss: 0.4117
Batch 46600/76266, Loss: 0.1812
Batch 46700/76266, Loss: 0.3478
Batch 46800/76266, Loss: 0.1095
Batch 46900/76266, Loss: 0.0072
Batch 47000/76266, Loss: 0.2987
Batch 47100/76266, Loss: 0.1336
Batch 47200/76266, Loss: 0.1327
Batch 47300/76266, Loss: 0.0167
Batch 47400/76266, Loss: 0.0725
Batch 47500/76266, Loss: 0.1877
Batch 47600/76266, Loss: 0.2367
Batch 47700/76266, Loss: 0.2654
Batch 47800/76266, Loss: 0.3625
Batch 47900/76266, Loss: 0.1568
Batch 48000/76266, Loss: 0.0108
Batch 48100/76266, Loss: 0.2208
Batch 48200/76266, Loss: 0.0482
Batch 48300/76266, Loss: 0.1040
Batch 48400/76266, Loss: 0.2383
Batch 48500/76266, Loss: 0.2394
Batch 48600/76266, Loss: 0.2567
Batch 48700/76266, Loss: 0.1696
Batch 48800/76266, Loss: 0.2048
Batch 48900/76266, Loss: 0.3336
Batch 49000/76266, Loss: 0.4101
Batch 49100/76266, Loss: 0.4949
Batch 49200/76266, Loss: 0.3964
Batch 49300/76266, Loss: 0.0247
Batch 49400/76266, Loss: 0.3318
Batch 49500/76266, Loss: 0.4048
Batch 49600/76266, Loss: 0.3136
Batch 49700/76266, Loss: 0.7093
Batch 49800/76266, Loss: 0.1183
Batch 49900/76266, Loss: 0.0693
Batch 50000/76266, Loss: 1.0703
Batch 50100/76266, Loss: 0.2095
Batch 50200/76266, Loss: 0.0469
Batch 50300/76266, Loss: 0.2126
Batch 50400/76266, Loss: 0.1214
Batch 50500/76266, Loss: 0.0419
Batch 50600/76266, Loss: 0.1212
Batch 50700/76266, Loss: 0.1864
Batch 50800/76266, Loss: 0.7854
Batch 50900/76266, Loss: 0.3590
Batch 51000/76266, Loss: 0.3629
Batch 51100/76266, Loss: 0.2144
Batch 51200/76266, Loss: 0.1720
Batch 51300/76266, Loss: 0.0129
Batch 51400/76266, Loss: 0.2420
Batch 51500/76266, Loss: 0.2328
Batch 51600/76266, Loss: 0.6196
Batch 51700/76266, Loss: 0.2302
Batch 51800/76266, Loss: 0.3909
Batch 51900/76266, Loss: 0.3737
Batch 52000/76266, Loss: 0.2714
Batch 52100/76266, Loss: 0.0635
Batch 52200/76266, Loss: 0.0602
Batch 52300/76266, Loss: 0.1212
Batch 52400/76266, Loss: 0.1357
Batch 52500/76266, Loss: 0.2878
Batch 52600/76266, Loss: 0.0861
Batch 52700/76266, Loss: 0.2705
Batch 52800/76266, Loss: 0.2293
Batch 52900/76266, Loss: 0.1402
Batch 53000/76266, Loss: 0.3323
Batch 53100/76266, Loss: 0.2738
Batch 53200/76266, Loss: 0.1740
Batch 53300/76266, Loss: 0.1859
Batch 53400/76266, Loss: 0.3540
Batch 53500/76266, Loss: 0.1993
Batch 53600/76266, Loss: 0.0559
Batch 53700/76266, Loss: 0.0640
Batch 53800/76266, Loss: 0.2934
Batch 53900/76266, Loss: 0.2527
Batch 54000/76266, Loss: 0.0169
Batch 54100/76266, Loss: 0.3299
Batch 54200/76266, Loss: 0.6973
Batch 54300/76266, Loss: 0.3558
Batch 54400/76266, Loss: 0.3831
Batch 54500/76266, Loss: 0.5005
Batch 54600/76266, Loss: 0.1608

Batch 54700/76266, Loss: 0.1896
Batch 54800/76266, Loss: 0.3433
Batch 54900/76266, Loss: 0.3368
Batch 55000/76266, Loss: 0.1522
Batch 55100/76266, Loss: 0.0804
Batch 55200/76266, Loss: 0.2225
Batch 55300/76266, Loss: 0.4172
Batch 55400/76266, Loss: 0.4092
Batch 55500/76266, Loss: 0.3974
Batch 55600/76266, Loss: 0.4849
Batch 55700/76266, Loss: 0.1898
Batch 55800/76266, Loss: 0.0591
Batch 55900/76266, Loss: 0.6237
Batch 56000/76266, Loss: 0.2535
Batch 56100/76266, Loss: 0.1147
Batch 56200/76266, Loss: 0.1646
Batch 56300/76266, Loss: 0.4874
Batch 56400/76266, Loss: 0.2206
Batch 56500/76266, Loss: 0.1172
Batch 56600/76266, Loss: 0.1064
Batch 56700/76266, Loss: 0.3625
Batch 56800/76266, Loss: 0.2980
Batch 56900/76266, Loss: 0.0953
Batch 57000/76266, Loss: 0.2911
Batch 57100/76266, Loss: 0.1530
Batch 57200/76266, Loss: 0.5460
Batch 57300/76266, Loss: 0.3300
Batch 57400/76266, Loss: 0.0995
Batch 57500/76266, Loss: 0.5230
Batch 57600/76266, Loss: 0.1243
Batch 57700/76266, Loss: 0.2001
Batch 57800/76266, Loss: 0.3771
Batch 57900/76266, Loss: 0.0801
Batch 58000/76266, Loss: 0.3355
Batch 58100/76266, Loss: 0.1427
Batch 58200/76266, Loss: 0.2683
Batch 58300/76266, Loss: 0.3012
Batch 58400/76266, Loss: 0.2153
Batch 58500/76266, Loss: 0.0044
Batch 58600/76266, Loss: 0.3268
Batch 58700/76266, Loss: 0.2245
Batch 58800/76266, Loss: 0.1616
Batch 58900/76266, Loss: 0.0234
Batch 59000/76266, Loss: 0.2683
Batch 59100/76266, Loss: 0.0541
Batch 59200/76266, Loss: 0.4166
Batch 59300/76266, Loss: 0.2229
Batch 59400/76266, Loss: 0.3467
Batch 59500/76266, Loss: 0.0295
Batch 59600/76266, Loss: 0.5903
Batch 59700/76266, Loss: 0.2985
Batch 59800/76266, Loss: 0.3161
Batch 59900/76266, Loss: 0.1396
Batch 60000/76266, Loss: 0.1219
Batch 60100/76266, Loss: 0.5371
Batch 60200/76266, Loss: 0.2132
Batch 60300/76266, Loss: 0.2418
Batch 60400/76266, Loss: 0.2414
Batch 60500/76266, Loss: 0.2302
Batch 60600/76266, Loss: 0.0516
Batch 60700/76266, Loss: 0.3553
Batch 60800/76266, Loss: 0.1893
Batch 60900/76266, Loss: 0.1057
Batch 61000/76266, Loss: 0.0401
Batch 61100/76266, Loss: 0.0511
Batch 61200/76266, Loss: 0.6577
Batch 61300/76266, Loss: 0.0884
Batch 61400/76266, Loss: 0.0192
Batch 61500/76266, Loss: 0.0366
Batch 61600/76266, Loss: 0.1017
Batch 61700/76266, Loss: 0.0812
Batch 61800/76266, Loss: 0.0407
Batch 61900/76266, Loss: 0.0406
Batch 62000/76266, Loss: 0.2847
Batch 62100/76266, Loss: 0.2727
Batch 62200/76266, Loss: 0.3334
Batch 62300/76266, Loss: 0.8223
Batch 62400/76266, Loss: 0.4784
Batch 62500/76266, Loss: 0.0650
Batch 62600/76266, Loss: 0.1525
Batch 62700/76266, Loss: 0.4488
Batch 62800/76266, Loss: 0.2304
Batch 62900/76266, Loss: 0.1453
Batch 63000/76266, Loss: 0.1157
Batch 63100/76266, Loss: 0.2676

Batch 63100/76266, Loss: 0.2030
Batch 63200/76266, Loss: 0.3617
Batch 63300/76266, Loss: 0.2131
Batch 63400/76266, Loss: 0.3294
Batch 63500/76266, Loss: 0.1130
Batch 63600/76266, Loss: 0.0462
Batch 63700/76266, Loss: 0.4896
Batch 63800/76266, Loss: 0.2398
Batch 63900/76266, Loss: 0.0293
Batch 64000/76266, Loss: 0.0673
Batch 64100/76266, Loss: 0.4444
Batch 64200/76266, Loss: 0.2059
Batch 64300/76266, Loss: 0.2487
Batch 64400/76266, Loss: 0.3448
Batch 64500/76266, Loss: 0.1600
Batch 64600/76266, Loss: 0.0186
Batch 64700/76266, Loss: 0.1524
Batch 64800/76266, Loss: 0.3208
Batch 64900/76266, Loss: 0.4733
Batch 65000/76266, Loss: 0.0206
Batch 65100/76266, Loss: 0.0453
Batch 65200/76266, Loss: 0.3941
Batch 65300/76266, Loss: 0.2382
Batch 65400/76266, Loss: 0.3884
Batch 65500/76266, Loss: 0.3369
Batch 65600/76266, Loss: 0.0572
Batch 65700/76266, Loss: 0.7853
Batch 65800/76266, Loss: 0.1477
Batch 65900/76266, Loss: 0.2853
Batch 66000/76266, Loss: 0.7495
Batch 66100/76266, Loss: 0.1974
Batch 66200/76266, Loss: 0.2281
Batch 66300/76266, Loss: 0.0230
Batch 66400/76266, Loss: 0.3482
Batch 66500/76266, Loss: 0.1285
Batch 66600/76266, Loss: 0.1924
Batch 66700/76266, Loss: 0.0904
Batch 66800/76266, Loss: 0.2120
Batch 66900/76266, Loss: 0.7522
Batch 67000/76266, Loss: 0.0765
Batch 67100/76266, Loss: 0.5757
Batch 67200/76266, Loss: 0.3699
Batch 67300/76266, Loss: 0.5817
Batch 67400/76266, Loss: 0.1048
Batch 67500/76266, Loss: 0.3491
Batch 67600/76266, Loss: 0.4767
Batch 67700/76266, Loss: 0.0318
Batch 67800/76266, Loss: 0.0393
Batch 67900/76266, Loss: 0.0787
Batch 68000/76266, Loss: 0.1504
Batch 68100/76266, Loss: 0.1515
Batch 68200/76266, Loss: 0.2222
Batch 68300/76266, Loss: 0.5996
Batch 68400/76266, Loss: 0.2644
Batch 68500/76266, Loss: 0.1202
Batch 68600/76266, Loss: 0.2455
Batch 68700/76266, Loss: 0.4373
Batch 68800/76266, Loss: 0.2593
Batch 68900/76266, Loss: 0.5122
Batch 69000/76266, Loss: 0.1005
Batch 69100/76266, Loss: 0.0634
Batch 69200/76266, Loss: 0.3922
Batch 69300/76266, Loss: 0.2152
Batch 69400/76266, Loss: 0.3640
Batch 69500/76266, Loss: 0.0211
Batch 69600/76266, Loss: 0.1999
Batch 69700/76266, Loss: 0.4093
Batch 69800/76266, Loss: 0.1706
Batch 69900/76266, Loss: 0.1291
Batch 70000/76266, Loss: 0.2971
Batch 70100/76266, Loss: 0.4176
Batch 70200/76266, Loss: 0.0084
Batch 70300/76266, Loss: 0.2578
Batch 70400/76266, Loss: 0.2648
Batch 70500/76266, Loss: 0.2190
Batch 70600/76266, Loss: 0.1975
Batch 70700/76266, Loss: 0.3772
Batch 70800/76266, Loss: 0.5109
Batch 70900/76266, Loss: 0.2435
Batch 71000/76266, Loss: 0.2286
Batch 71100/76266, Loss: 0.3751
Batch 71200/76266, Loss: 0.1805
Batch 71300/76266, Loss: 0.2184
Batch 71400/76266, Loss: 0.4940
Batch 71500/76266, Loss: 0.1573

Batch 71600/76266, Loss: 0.2094
 Batch 71700/76266, Loss: 0.1725
 Batch 71800/76266, Loss: 0.0693
 Batch 71900/76266, Loss: 0.0429
 Batch 72000/76266, Loss: 0.1038
 Batch 72100/76266, Loss: 0.3825
 Batch 72200/76266, Loss: 0.1193
 Batch 72300/76266, Loss: 0.0648
 Batch 72400/76266, Loss: 0.4082
 Batch 72500/76266, Loss: 0.0411
 Batch 72600/76266, Loss: 0.2359
 Batch 72700/76266, Loss: 0.1867
 Batch 72800/76266, Loss: 0.1440
 Batch 72900/76266, Loss: 0.2207
 Batch 73000/76266, Loss: 0.2421
 Batch 73100/76266, Loss: 0.1595
 Batch 73200/76266, Loss: 0.1041
 Batch 73300/76266, Loss: 0.0301
 Batch 73400/76266, Loss: 0.1355
 Batch 73500/76266, Loss: 0.5367
 Batch 73600/76266, Loss: 0.0675
 Batch 73700/76266, Loss: 0.2984
 Batch 73800/76266, Loss: 0.1190
 Batch 73900/76266, Loss: 0.2122
 Batch 74000/76266, Loss: 0.2114
 Batch 74100/76266, Loss: 0.4029
 Batch 74200/76266, Loss: 0.3828
 Batch 74300/76266, Loss: 0.7851
 Batch 74400/76266, Loss: 0.2886
 Batch 74500/76266, Loss: 0.0401
 Batch 74600/76266, Loss: 0.2756
 Batch 74700/76266, Loss: 0.1328
 Batch 74800/76266, Loss: 0.5054
 Batch 74900/76266, Loss: 0.1643
 Batch 75000/76266, Loss: 0.2104
 Batch 75100/76266, Loss: 0.1821
 Batch 75200/76266, Loss: 0.1057
 Batch 75300/76266, Loss: 0.2152
 Batch 75400/76266, Loss: 0.5811
 Batch 75500/76266, Loss: 0.2578
 Batch 75600/76266, Loss: 0.4796
 Batch 75700/76266, Loss: 0.3168
 Batch 75800/76266, Loss: 0.4105
 Batch 75900/76266, Loss: 0.2084
 Batch 76000/76266, Loss: 0.0578
 Batch 76100/76266, Loss: 0.0474
 Batch 76200/76266, Loss: 0.0234

Epoch 1 completed in 6015.94s. Average Training Loss: 0.2668

Transformer training finished.

Saving fine-tuned Transformer model to /content/drive/My Drive/Colab Notebooks/SentimentModels_Stars...

Model and tokenizer saved successfully.

Evaluating Transformer model...

Transformer Classification Report (Stars):

	precision	recall	f1-score	support
Negative (1-2 Stars)	0.8698	0.9041	0.8866	67186
Neutral (3 Stars)	0.5741	0.4353	0.4951	26626
Positive (4-5 Stars)	0.9543	0.9715	0.9628	211253
accuracy			0.9098	305065
macro avg	0.7994	0.7703	0.7815	305065
weighted avg	0.9025	0.9098	0.9052	305065

Generating Confusion Matrix for Transformer...

Confusion matrix saved to /content/drive/My Drive/Colab Notebooks/SentimentModels_Stars/transformer_confusion_matrix_stars.png

--- Script Finished ---

```

# Import necessary libraries
import torch
import re
import os
from transformers import AutoTokenizer, AutoModelForSequenceClassification
import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from nltk.stem import WordNetLemmatizer

# --- Ensure NLTK data is available (run once if needed) ---
def download_nltk_data():
    """Downloads necessary NLTK data packages."""
    packages = ['punkt', 'stopwords', 'wordnet']
    for package in packages:
        try:
            nltk.data.find(f'corpora/{package}' if package != 'punkt' else f'tokenizers/{package}')
        except LookupError:
            print(f"Downloading NLTK package: {package}...")
            nltk.download(package, quiet=True)
    print("NLTK data check complete.")

download_nltk_data()

# --- Configuration ---
# !! IMPORTANT !! Make sure this matches the directory where you saved
# the star-rating-trained model in the previous script.
try:
    # If running in Colab and saved to Drive
    from google.colab import drive
    # Ensure Drive is mounted if needed (might not be necessary if already mounted)
    try:
        drive.mount('/content/drive', force_remount=True)
    except: # Handle cases where drive might already be mounted or not in Colab
        pass
    model_save_directory = '/content/drive/My Drive/Colab Notebooks/SentimentModels_Stars'
    print("Assuming Colab environment for model path.")
except ModuleNotFoundError:
    # If running locally
    model_save_directory = './SentimentModels_Stars'
    print("Assuming local environment for model path.")

print(f"Attempting to load model from: {model_save_directory}")

# Check for GPU
device = torch.device('cuda') if torch.cuda.is_available() else torch.device('cpu')
print(f"Using device: {device}")

# Define the mapping from integer labels (used during training) to sentiment strings
# Must match the mapping used when training the star-based model:
# 0: Negative (1-2 Stars), 1: Neutral (3 Stars), 2: Positive (4-5 Stars)
label_map = {
    0: 'Negative',
    1: 'Neutral',
    2: 'Positive'
}

# --- Text Preprocessing Function (Same as before) ---
lemmatizer = WordNetLemmatizer()
stop_words_set = set(stopwords.words('english'))

def preprocess_text(text):
    """Cleans and preprocesses text data."""
    if not isinstance(text, str):
        return ""
    text = text.lower() # Lowercase
    text = re.sub(r'http\S+|www\S+|https\S+', '', text, flags=re.MULTILINE) # Remove URLs
    text = re.sub(r'@\w+|#', '', text) # Remove mentions and hashtags
    text = re.sub(r'[^\w\s]', '', text) # Remove punctuation
    text = re.sub(r'\d+', '', text) # Remove numbers
    tokens = word_tokenize(text) # Tokenize
    processed_tokens = [
        lemmatizer.lemmatize(word) for word in tokens
        if word not in stop_words_set and word.isalpha() and len(word) > 1
    ]
    return ' '.join(processed_tokens)

```

```

# --- Load Model and Tokenizer ---
try:
    print("Loading fine-tuned tokenizer...")
    tokenizer = AutoTokenizer.from_pretrained(model_save_directory)
    print("Loading fine-tuned model...")
    model = AutoModelForSequenceClassification.from_pretrained(model_save_directory)
    model.to(device) # Move model to the appropriate device
    model.eval() # Set model to evaluation mode
    print("Model and tokenizer loaded successfully.")
    model_loaded = True
except OSError as e:
    print(f"Error loading model/tokenizer from {model_save_directory}: {e}")
    print("Please ensure the directory exists and contains the saved model files ('pytorch_model.bin', 'config.json', 'tokenizer_config.json'")
    print("Cannot proceed with predictions.")
    model_loaded = False
except Exception as e:
    print(f"An unexpected error occurred during loading: {e}")
    model_loaded = False

# --- Prediction Function ---
def predict_overall_sentiment(text, loaded_model, loaded_tokenizer):
    """Predicts overall sentiment for a given text using the star-rating model."""
    if not model_loaded:
        return "Error: Model not loaded."

    # 1. Preprocess the input text
    processed_text = preprocess_text(text)
    if not processed_text:
        return "Input text is empty after preprocessing."

    # 2. Tokenize
    # Adjust max_length if needed, should match training if possible
    inputs = loaded_tokenizer(
        processed_text,
        return_tensors='pt',
        truncation=True,
        padding=True,
        max_length=256 # Use the same MAX_LEN as during training if possible
    )

    # 3. Move inputs to the same device as the model
    inputs = {k: v.to(device) for k, v in inputs.items()}

    # 4. Predict
    with torch.no_grad(): # Disable gradient calculations for inference
        outputs = loaded_model(**inputs)
        logits = outputs.logits

```

